PROJECT DESCRIPTION · ALEXA+ TRACK

AuraAssist

A context-aware, hands-free home companion for Alexa+



Event
Amazon "Build, Ship, Shape" Developer Hackathon

Tracks
Alexa+ · AWS Builder Mini Challenge

Repository
github.com/Alagapie/aura-assist

01 Executive Summary

AuraAssist makes a smart home easier to understand and run. It is designed for people with low vision, people with limited mobility, older adults and family caregivers.

Traditional assistants expect users to remember device names and issue one command per action. With AuraAssist, a user can ask what is on a table, ask where an item was last seen and get non-visual directions to it, ask what changed today, or ask the assistant to prepare the home for bed. The assistant looks up the home's context, chooses the right tools, reads back a plan before acting on anything significant, and reports the result in short, spoken language.

9	33	3	6 + 101
MCP tools over Streamable HTTP	simulated devices in 5 rooms	Nova Lite roles: tools, vision, briefing	smoke suites + simulator checks, all passing

The working prototype combines a **self-hosted MCP server** (Streamable HTTP at `/mcp`, protocol `2026-07-28`, official MCP Python SDK 2.3), **Amazon Nova Lite on Amazon Bedrock**, a **3D web home simulator** that is both the house and its cameras, and a **SQLite event store**. It also runs in a deterministic offline mode with the same tool contract, so the demo never depends on network access.

02 The Problem

A smart home is hard to use when it depends on visual dashboards, exact device names or a chain of individual commands. For someone with low vision or limited mobility, knowing that a light *can* be switched on is not enough. They also need to know what is in a room, find a misplaced object, check that the front door is locked, or review the day without navigating screens.

> “What's on the coffee table?” · “Where are my glasses?” · “Which lights are still on?” · “What changed in the house today?” · “Prepare the house for bed.”

AuraAssist brings environmental context, device state, event history and routines into one conversation.

03 The Solution

AuraAssist is a conversational layer over a simulated smart home. **All reading and control of the home goes through the MCP server.** The in-app agent, the 3D web app and any MCP client (Alexa+, MCP Inspector) share the same tools, the same state and the same event log.

When a user speaks or types, Amazon Nova Lite, through the Bedrock Converse API with tools, decides which MCP tools to call. The server runs them, checks safety rules in code, updates the home and streams the change back to the 3D house live.

04 Core Capabilities

4.1 Scene description

Every room in the 3D house works as a camera. The simulator sends a whole-room frame plus four zoomed tiles, and Nova Lite turns them into structured objects, each with the surface it rests on, a confidence score, and any hazards. If a camera hasn't seen a room yet, or the vision call fails, AuraAssist says so instead of guessing. The fridge has no camera feed, so its contents come from a simulated scene.

4.2 Object finding with memory

Every recognised object is stored as a text sighting, such as “phone, on the coffee table, 12:45”. “Where are my keys?” checks the live camera views first, then sighting history, and answers with the location and the time last seen.

4.3 Non-visual guidance for people with low vision

Once an object is found, `guide_to_object` gives directions that don't rely on sight: which surface it is on and how to tell that surface apart using landmarks (window, door, which side of the bed), what it sits next to, its height, and a walking route from the room's door. It deliberately avoids “left” and “right”, which depend on which way the person is facing.

4.4 Home-state awareness

33 devices across the living room, bedroom, kitchen, bathroom and hallway: lights, doors, curtains, fans, air conditioners with temperature control, a TV, fridge, cooker, shower, front-door lock, doorbell and a sensor. “What's happening?” returns a short spoken summary grouped by security, climate, lights and curtains, put together in code from verified state.

4.5 Routines: plan, approve, execute

Bedtime, leaving-home and morning routines first return a plan, such as “turn off 6 lights and close the curtains”, including exceptions like an unlocked front door. They run only after the user says yes. A pending plan is cleared once the conversation moves on, so a later “yes” can never approve a stale plan.

4.6 Event timeline and daily briefing

The event log records device changes, routine runs and object sightings. `get_recent_events` groups repeated events into a short summary, and `generate_daily_briefing` summarises the day in two calm sentences, rewritten by Nova Lite.

4.7 Consent-gated caregiver summary

A family caregiver gets a summary only when the resident has set the consent preference. The summary is deliberately narrow: routines, the front door and safety alerts. It never includes camera content, object sightings or guesses about health or mood. Without consent, the request is refused and the refusal is recorded in the log.

4.8 Voice-first interaction

Push-to-talk, a wake word and spoken replies through the browser's Web Speech API (Chrome or Edge). Typing works in any browser. Replies are short and conversational by design.

05 How It Works

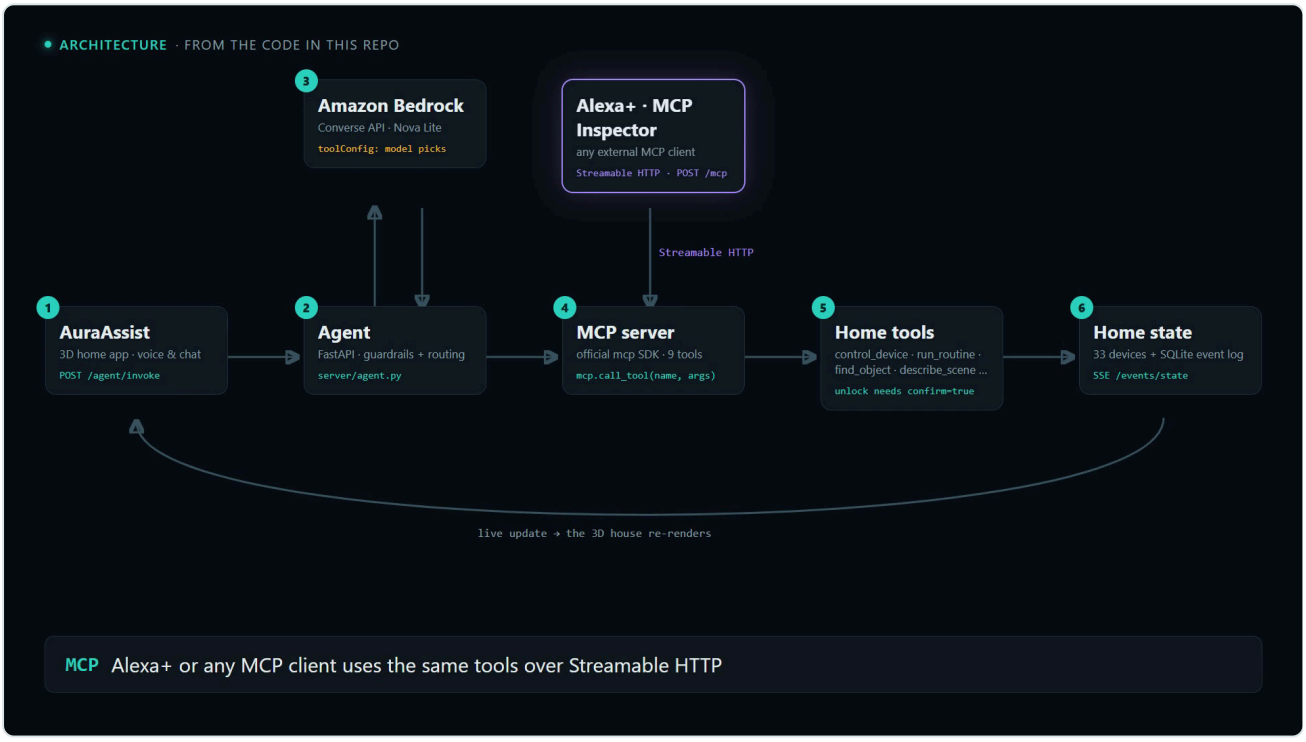


Figure 1. Architecture, drawn from the code: AuraAssist → Agent → Amazon Bedrock (Nova Lite) → MCP server → home tools → home state, with live updates back to the 3D house. External MCP clients connect over Streamable HTTP.

1. The user speaks or types a request in the 3D simulator.
2. The simulator sends it to the AuraAssist server (`POST /agent/invoke`).
3. The agent passes the request and the 9 MCP tool definitions to **Amazon Nova Lite** through the Bedrock **Converse API** (`toolConfig`). The model chooses which tools to call.
4. Each tool call runs through the **MCP server's own dispatch**, the same tools external MCP clients reach over Streamable HTTP.
5. The tools read or change home state, record events, or run vision on the latest camera frame.
6. The change streams back to the 3D house live (Server-Sent Events), and the user hears a short reply.

7 Unlocking and routine execution are downgraded to “plan only” by code unless the user's current message is an explicit approval.

If Bedrock is unreachable, the agent falls back to the offline engine, so the home keeps working.



Figure 2. A real Bedrock turn. “Turn off the living room lights”: Nova Lite chose six `control_device` calls through the MCP server, the state changed from 6 lights on to 3, and the 3D house updated live.

06 Technical Architecture

Component	Role
3D web simulator	React and Three.js. The house, its room cameras, and the chat and voice UI. Includes a browser MCP client that shows the raw handshake and tool calls.
Agent	FastAPI (`server/agent.py`). Sends requests to Nova Lite with the MCP tools and enforces safety rules in code.
MCP server	`server/mcp_tools.py` : 9 tools over Streamable HTTP at `/mcp` , protocol `2026-07-28` .
Amazon Bedrock · Nova Lite	Choosing tools (Converse + `toolConfig`), multi-image vision, daily briefing.
Home-state service	33 simulated devices, with live updates to the simulator.
Event store	SQLite: device changes, routine runs, object sightings, briefings.
Routine planner	Bedtime, leaving-home and morning plans; plan → approve → execute.
Offline engine	Deterministic, with the same tool contract, for tests and offline demos.

MCP tools

`get_home_snapshot` `control_device` `run_routine` `get_recent_events` `find_object` `guide_to_object` `describe_scene` `generate_daily_briefing` `set_user_preference`

Verification

Six smoke suites cover MCP over Streamable HTTP, agent behaviour and safety rules, vision, guidance, caregiver consent and object finding end to end. There are also 101 simulator checks. All pass on a fresh clone.

07 Safety, Privacy and Responsible Design

- **No facial recognition or identification.** Vision reports objects and hazards. For people, it reports only a count.
- **Images are not stored.** The latest frame from each camera is kept in server memory so it can be analysed. It is replaced by the next frame, cleared on reset or restart, and never written to disk. Frames are sent to Amazon Bedrock for analysis.
- **Only text is kept.** Sightings and events are stored in a local SQLite file (`data/aura.db`) on the machine running the server. The prototype has no automatic retention limit yet.
- **Confirmation in code, not the prompt.** Unlocking needs `confirm=true` and an explicit approval in the current message. Routines always show their plan first.
- **Honest about uncertainty and failure.** Objects carry confidence scores. Unseen rooms are reported as unseen. A failed action is reported as failed, never as done.
- **Consent for caregivers.** Nothing is shared without the resident's consent, and camera content is never shared.
- **Scope.** An assistance prototype. Not an emergency-response, medical or fall-detection system.

08 Intended Users and Impact

Intended users	Potential benefit
People with low vision	Spoken scene descriptions, object finding, and non-visual directions to the object.
People with limited mobility	One request runs a whole routine instead of many separate commands.
Older adults	Simple spoken checks of the home and what changed.
Family caregivers	Narrow, consent-gated summaries of home activity.
Smart-home users	Contextual questions and coordinated control across many devices.

09 What Makes AuraAssist Different

- **Everything goes through MCP.** The model chooses real MCP tools, so the agent, the web app and any MCP client go through one set of tools, one state and one event log.

- **Safety and honesty are enforced in code.** Nothing depends on how well the model follows instructions.
- **Seeing turns into memory.** Camera views become text sightings, which become directions a blind or low-vision user can follow.

10 Current Prototype and Future Direction

Runs end to end today

- MCP server over Streamable HTTP
- Nova Lite: tools, vision, briefing
- 3D simulator with live state
- Routines with approval
- Object finding + non-visual guidance
- Caregiver consent, browser voice

Simulated

- The home and its 33 devices (no physical hardware)
- The fridge camera's contents

Not yet connected

- A physical Alexa+ device
- Commercial smart-home platforms

The MCP server is the integration point for an Alexa+-style client, shown here with the web simulator and standard MCP clients.

Next steps

- Connect to real smart-home platforms where APIs and permissions allow.
- Add a retention setting for the event log, and per-camera privacy controls.
- Improve accessibility settings, spoken feedback and multilingual support.
- Deploy the MCP server for remote Alexa+ clients.

11 Conclusion

AuraAssist shows how an MCP-based assistant on Amazon Bedrock can make a smart home more accessible. It combines scene understanding, object memory, non-visual guidance, home-state awareness and confirmation-gated routines in one conversation. It helps people understand their surroundings, not just issue device commands, and it handles privacy, safety and honesty in code.